

复习

第一章 计算机网络概述

一、Internet

1. 从组成角度理解

Internet 是一个由大量计算设备、通信链路和分组交换设备互连形成的全球性网络。不是一张单独的网络，而是很多网络互连形成的“网络的网络”。

2. 从服务角度理解

Internet 是一个为分布式应用程序提供通信服务的基础设施。

Internet与internet的区别：

- internet：泛指多个网络互连形成的网络；
- Internet：特指全球公共互联网。

二、协议

1. 协议的定义

网络协议是通信双方在进行数据交换时共同遵守的规则集合。

2. 协议三要素

（1）语法 Syntax

规定数据和控制信息的结构与格式。

（2）语义 Semantics

规定各字段表示什么含义，以及接收到信息后应执行什么操作。

（3）时序 Timing

规定信息交换的先后顺序、发送速度和时间关系。

三、网络边缘

1. 网络边缘是什么

网络边缘是互联网中运行应用程序、产生和接收数据的部分。

它主要由各种端系统组成。

2. 网络边缘的作用

网络边缘主要负责：

- 运行网络应用；
- 产生数据；
- 接收数据；
- 完成用户或进程之间的通信。

因此：

应用程序运行在网络边缘，而不是网络核心。

3. 网络边缘中的通信方式

客户—服务器模式

客户端主动请求服务，服务器提供服务。

特点：

- 服务器通常持续在线；
- 服务器地址相对固定；
- 客户端之间一般不直接通信；
- 大量服务器通常部署在数据中心。

对等模式 P2P

端系统之间可以直接通信，每个节点既可以是客户端，也可以是服务器。

特点：

- 节点之间直接交换数据；
- 扩展性较好；
- 节点可能随时上线或离线；
- 管理和安全较复杂。

四、接入网

1. 接入网是什么

接入网是将端系统连接到第一个路由器，也就是边缘路由器的网络。

整体路径可以理解为：

端系统 → 接入网络 → 边缘路由器 → 网络核心

接入网解决的问题是：

2. 常见接入网络

家庭接入

- 家庭Wi-Fi。

企业和校园接入

- 无线局域网；
- 校园网交换机；
- 企业内部网络。

移动接入

- 5G；
- 无线基站。

3. 物理媒体

物理媒体是信号传播所经过的实际介质。

导引型媒体

信号沿着固体介质传播。

非导引型媒体

信号通过空气或空间传播。

五、网络核心

1. 网络核心是什么

网络核心由大量路由器和互联链路组成。

主要功能是：

- 为网络边缘提供连通性；
- 对分组进行转发；
- 选择传输路径。

2. 路由选择与转发

路由选择

决定数据从源主机到目的主机应经过哪条路径。

分组转发

路由器根据转发表，把收到的分组从某个输入端口送到正确的输出端口。

3. 电路交换

电路交换在通信之前，先建立一条端到端的专用通信路径。

一般包括三个阶段：

1. 建立连接；
2. 数据传输；
3. 释放连接。

特点

- 通信前需要建立连接；
- 资源被预留；
- 用户独占一定带宽；
- 通信质量比较稳定；
- 没有数据发送时，预留资源仍被占用；
- 适合持续、稳定的数据通信。

4. 电路交换中的复用

频分复用 FDM

将链路总频带划分为多个互不重叠的频段，不同用户使用不同频段。

特点：

- 多个用户可以同时传输；
- 用户占用不同频率范围。

时分复用 TDM

将时间划分为多个时隙，不同用户轮流占用信道。

特点：

- 用户使用同一频带；
- 不同用户占用不同时间段。

5. 分组交换

分组交换先把长报文划分为多个较小的分组，再逐个发送。

每个分组通常包含：

- 首部；
- 数据部分。

首部中包含目的地址等控制信息。

特点

- 不预留专用链路；
- 多个用户共享链路；
- 资源利用率较高；
- 适合突发性数据；
- 可能产生排队；
- 可能发生丢包；
- 分组可能经过不同路径。

互联网的网络核心主要采用分组交换。

6. 存储转发

路由器采用存储转发方式：

路由器必须完整接收一个分组后，才能将其转发到下一条链路。

7. 分组交换与电路交换对比

比较项	电路交换	分组交换
是否建立专用连接	是	一般不需要
资源使用	预留、独占	动态共享
利用率	突发业务下较低	较高
时延	较稳定	可能变化
排队与丢包	通常较少	可能发生
适用业务	连续稳定业务	突发数据业务

核心结论：

电路交换保证性较强，分组交换资源利用率较高。

六、Internet与ISP结构

1. ISP是什么

ISP是互联网服务提供商，它向用户、企业或其他网络提供互联网连接服务。

2. 互联网为什么需要ISP

互联网由大量独立网络组成，这些网络需要通过ISP进行互连。

普通用户通常不能直接连接全球互联网，而是先接入某个ISP。

例如：

家庭用户 → 接入ISP → 区域ISP → 骨干ISP → 目标网络

3. ISP的层次结构

接入ISP

直接连接家庭、学校和企业用户。

例如：

- 家庭宽带；
- 校园网；
- 移动网络。

区域ISP

连接多个接入ISP，并将流量送往更高层网络。

一级ISP或全球ISP

拥有大规模骨干网络，在不同地区和国家之间传送数据。

4. 对等互连

不同ISP之间不一定都通过上级ISP通信，也可以直接互连。

常见方式包括：

- 对等连接 Peering；
- 互联网交换中心 IXP；

大型互联网公司可能建设自己的全球网络和数据中心，从而减少对传统ISP的依赖。

5. 网络之网络

互联网没有一个唯一的中心，也不是由某个单一机构控制。

它的本质是：

大量ISP、企业网络、校园网、数据中心网络和家庭网络相互连接形成的网络之网络。

七、网络性能

1. 节点总时延

一个分组在网络节点上经历的总时延包括：

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

2. 处理时延

路由器处理分组所需时间。

主要包括：

- 检查首部；
- 检查比特差错；
- 查询转发表；
- 确定输出端口。

3. 排队时延

分组进入路由器后，可能需要在输出队列中等待。

排队时延与网络负载有关。

流量强度

设：

- (L) ：平均分组长度；
- (a) ：分组平均到达速率；
- (R) ：链路传输速率。

流量强度为：

$$\frac{La}{R}$$

判断：

- (La/R) 很小：排队时延较小；
- (La/R) 接近1：排队时延迅速增大；
- $(La/R > 1)$ ：到达速度超过发送能力，队列不断增长。

4. 发送时延

发送时延也叫传输时延，是将整个分组推入链路所需的时间。

$$d_{\text{trans}} = \frac{L}{R}$$

其中：

- (L): 分组长度, 单位必须是bit;
- (R): 链路速率, 单位是bit/s。

5. 传播时延

传播时延是信号从链路一端传播到另一端所需的时间。

$$d_{\text{prop}} = \frac{D}{V}$$

其中:

- (D): 链路长度;
- (V): 信号传播速度。

6. 丢包

路由器的缓存容量有限。缓存被占满, 新到达的分组就会被丢弃。

7. 吞吐量

吞吐量表示单位时间内, 发送方与接收方之间实际成功传输的数据量。

端到端吞吐量由最慢链路决定。

最慢的链路称为: 瓶颈链路

带宽与吞吐量区别

- 带宽: 链路理论上的最大传输能力;
- 吞吐量: 实际获得的传输速率。

八、协议层次与服务模型

1. 为什么要分层

计算机网络功能非常复杂, 如果所有功能混在一起, 系统将难以设计和维护。

分层的好处:

- 将复杂问题分解;
- 每层只完成特定功能;
- 上层使用下层服务;
- 各层可以相对独立修改;
- 便于标准化和实现。

2. 协议与服务

协议

同一层不同设备中的对等实体之间遵守的规则。

属于水平关系。

服务

下层向上层提供的功能。

属于垂直关系。

3. OSI七层模型

从上到下：

1. 应用层；
2. 表示层；
3. 会话层；
4. 运输层；
5. 网络层；
6. 数据链路层；
7. 物理层。

OSI主要用于理论分析和教学。

4. TCP/IP四层模型

1. 应用层；
2. 运输层；
3. 网际层；
4. 网络接口层。

5. 五层教学模型

1. 应用层；
2. 运输层；
3. 网络层；
4. 数据链路层；
5. 物理层。

应用层

为网络应用提供服务。

数据单位一般称为报文。

运输层

负责进程到进程的通信。

主要功能：

- 端到端传输；
- 可靠传输；
- 流量控制；
- 拥塞控制；
- 端口寻址。

数据单位一般为：

- TCP报文段；
- UDP用户数据报。

网络层

负责将分组从源主机传到目的主机。

主要功能：

- 路由选择；
- 分组转发；
- IP寻址。

数据单位为数据报或分组。

数据链路层

负责相邻节点之间的数据传输。

主要功能：

- 成帧；
- 差错检测；
- 介质访问控制；
- MAC地址处理。

数据单位为帧。

物理层

负责在物理介质上传输比特。

数据单位为比特。

6. 封装与解封装

发送端封装

数据从上往下传递，每一层增加本层首部。

应用层数据 → 运输层报文段 → 网络层数据报 → 数据链路层帧 → 物理层比特

接收端解封装

数据从下往上传递，每一层读取并去掉相应首部。

比特 → 帧 → 数据报 → 报文段 → 应用数据

对等层原则

发送端第 (n) 层添加的首部，由接收端第 (n) 层读取。

8. 主机与路由器处理层次

主机

源主机和目的主机通常运行完整的五层协议栈。

路由器

路由器主要处理：

- 物理层；
- 数据链路层；
- 网络层。

第二章 应用层

2.1 网络应用原理

一、应用程序运行在哪里

网络应用程序运行在**网络边缘的端系统**中。

网络核心中的路由器通常只负责分组转发，不运行用户应用。因此开发网络应用时，主要编写端系统上的程序，不需要为网络核心设备编写应用程序。

不同主机上的应用进程通过网络交换报文进行通信，进程通过Socket使用运输层提供的服务。

一个网络进程通常由以下组合标识：

IP地址 + 端口号

- IP地址定位主机；
- 端口号定位主机中的具体应用进程。

二、B/S体系结构

B/S（Browser/Server，浏览器—服务器）是客户—服务器体系结构的一种典型形式。

1. 服务器特点

- 通常持续在线；
- 一般具有固定的IP地址；
- 为多个客户提供服务；
- 大型服务器通常部署在数据中心；

2. 客户端特点

- 主动向服务器发起请求；
- 可以间歇性接入网络；
- 客户端之间通常不直接通信；

3. 优缺点

优点：

- 集中管理方便；
- 数据和服务容易统一维护；
- 安全策略较容易实施。

缺点：

- 服务器可能成为性能瓶颈；
- 服务器故障可能影响大量用户；
- 用户增加时需要扩展服务器能力。

三、P2P体系结构

P2P即 Peer-to-Peer，对等体系结构。

1. 基本特点

- 没有必须持续在线的中心服务器；
- 任意端系统之间可以直接通信；
- 每个对等方既可以请求服务，也可以提供服务；

2. 自扩展性

P2P具有较强的**自扩展性**：

新节点加入时，既带来新的服务需求，也带来新的上传能力和资源。

因此用户增加不一定只增加服务器压力。

3. 缺点

- 节点状态不稳定；
- 数据和资源分散；
- 系统管理复杂；

四、混合结构

使用中心服务器完成用户查找或地址登记，真正的数据通信在两个用户之间直接进行。

2.2 Web与HTTP

一、Web的基本概念

一个Web页面通常由多个对象组成，每个对象通过URL进行标识。

URL的一般结构为：

协议://主机:端口/路径

HTTP默认端口是：80

二、HTTP的基本特点

HTTP是Web使用的应用层协议，采用客户—服务器方式工作。

- 浏览器是HTTP客户端；
- Web服务器是HTTP服务器；
- HTTP通常使用TCP提供可靠传输；
- HTTP通过请求报文和响应报文交换信息。

HTTP本身是**无状态协议**：

服务器默认不保存客户以前请求的状态，每一次请求原则上都被独立处理。

无状态可以简化服务器设计，但也导致服务器不能仅依靠HTTP自身记住用户身份等信息，因此需要Cookie。

三、HTTP基本交互过程

一次基本HTTP访问过程如下：

第一步：确定服务器地址

第二步：建立TCP连接

第三步：发送HTTP请求报文

第四步：服务器处理请求

第五步：返回HTTP响应

第六步：保持或关闭TCP连接

四、非持续HTTP与持续HTTP

1. 非持续HTTP

每个TCP连接最多传输一个Web对象，传输完成后关闭连接。

如果一个网页包含一个HTML文件和10张图片，就可能需要多次建立TCP连接。

获取一个对象的响应时间通常为：

$$2RTT + \text{对象传输时间}$$

其中：

- 一个RTT用于建立TCP连接；
- 一个RTT用于发送HTTP请求并等待响应；
- 再加上对象本身的传输时间。

缺点：

- 每个对象都需要重复建立连接；
- 增加时延；
- 增加服务器和操作系统开销。

HTTP/1.0通常采用非持续连接。

2. 持续HTTP

多个对象可以通过同一条TCP连接传输。

特点：

- 服务器返回一个对象后不立即关闭连接；
- 后续请求和响应继续使用原连接；
- 减少TCP连接建立次数；
- 降低时延和资源开销。

HTTP/1.1默认采用持续连接。

持续HTTP又可分为：

- 非流水线：收到上一个响应后再发送下一个请求；
- 流水线：不等待前一个响应，就连续发送多个请求。

四、HTTP报文结构

HTTP报文主要分为两类：

- **HTTP请求报文**：由客户端发送给服务器；
- **HTTP响应报文**：由服务器发送给客户端。

二者都是使用普通文本表示的，报文中每一个字段的值都是ASCII码串，基本组成是：

起始行 + 首部行 + 实体主体

1. HTTP请求报文

HTTP请求报文的一般结构为：

```
请求行
首部行1
首部行2
.....
空行
实体主体
```

例如：

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Connection: close
Accept-Language: zh-CN
```

2. 请求行

请求报文的第一行称为**请求行**，由三部分组成：

请求方法 + 请求对象 + HTTP版本

例如：

```
GET /index.html HTTP/1.1
```

3. 常见请求方法

GET

用于请求服务器上的资源。

特点：

- 请求参数可以放在URL中；
- 通常没有实体主体。

POST

用于向服务器提交数据。

例如：

- 提交登录表单；
- 上传用户信息；

POST提交的数据通常放在请求报文的**实体主体**中：

```
POST /login HTTP/1.1
Host: www.example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 25

username=alice&password=123
```

HEAD

HEAD与GET类似，但服务器只返回响应首部，不返回请求对象本身。

用途：

- 检查对象是否存在；
- 查看对象修改时间；
- 检查缓存副本是否有效。

PUT

用于将实体主体中的内容上传到指定路径。

如果该路径已有资源，通常表示替换；如果没有，则可能创建新资源。

DELETE

用于删除URL所标识的资源。

4. 请求首部行

请求行之后是若干个首部行，格式为：

```
字段名： 字段值
```

5. 请求实体主体

请求首部结束后必须有一个空行，空行之后才是实体主体。

实体主体主要用于携带客户端提交给服务器的数据，例如：

- 用户名和密码；
- 表单内容；
- JSON数据；

GET请求通常没有实体主体，POST和PUT请求通常带有实体主体。

五、HTTP响应报文

HTTP响应报文的一般结构为：

```
状态行
首部行1
首部行2
.....
空行
实体主体
```

例如：

```
HTTP/1.1 200 OK
Date: Sun, 12 Jul 2026 10:00:00 GMT
Server: Apache
Content-Length: 1250
Content-Type: text/html
Connection: close

<html>
.....
</html>
```

1. 状态行

响应报文第一行称为**状态行**，包含：

HTTP版本 + 状态码 + 状态短语

例如：

```
HTTP/1.1 200 OK
```

2. 常见状态码

200 OK

表示请求成功，请求的对象通常包含在响应实体主体中。

```
HTTP/1.1 200 OK
```

301 Moved Permanently

表示请求的资源已经永久移动到新地址。

新地址通过 `Location` 首部给出：

```
HTTP/1.1 301 Moved Permanently
Location: http://www.newsite.com/index.html
```

浏览器随后会请求新地址。

304 Not Modified

表示服务器上的对象自指定时间或指定ETag以来没有改变。

```
HTTP/1.1 304 Not Modified
```

响应中通常不携带对象实体，因为客户端或缓存已经保存了有效副本。

400 Bad Request

表示服务器无法理解请求报文，可能是请求格式错误。

```
HTTP/1.1 400 Bad Request
```

404 Not Found

表示服务器上不存在请求的对象。

```
HTTP/1.1 404 Not Found
```

505 HTTP Version Not Supported

表示服务器不支持请求中使用的HTTP版本。

```
HTTP/1.1 505 HTTP Version Not Supported
```

3. 状态码分类

状态码范围	含义
1xx	提示信息
2xx	请求成功
3xx	重定向或缓存相关
4xx	客户端请求错误
5xx	服务器错误

4. 响应实体主体

响应报文的实体主体通常包含客户所请求的实际对象，但是并非所有响应都有实体主体。例如：

```
HTTP/1.1 304 Not Modified
```

通常不携带对象，因为客户端已有可用副本。

五、Cookie

1. Cookie的作用

HTTP本身无状态，而Cookie可以帮助网站维持用户状态。

常见用途：

- 用户登录和身份识别；
- 个性化推荐；
- 记录用户偏好；

2. Cookie的四个组成部分

1. HTTP响应报文中的 `Set-Cookie` 首部；
2. 后续HTTP请求报文中的 `Cookie` 首部；
3. 浏览器在用户设备中保存的Cookie文件；
4. Web网站的后端数据库。

3. 工作过程

用户第一次访问网站：

1. 浏览器发送普通HTTP请求；
2. 服务器为用户生成唯一标识符；
3. 服务器在后端数据库中建立对应记录；
4. 响应报文携带：

Set-Cookie: 1678

5. 浏览器保存该Cookie。

用户以后再次访问时，请求报文中携带：

Cookie: 1678

服务器根据该编号查询后端数据库，从而识别用户并恢复相关状态。

4. 核心理解

HTTP仍然是无状态协议，Cookie是在HTTP报文和服务器数据库的配合下额外维持用户状态。

Cookie本身通常只是一个标识或少量状态数据，并不一定直接保存用户的全部信息。

六、Web缓存

Web缓存也称为代理服务器或Web Cache。

1. 作用

目标是尽量不访问源服务器，直接满足客户的请求。

Web缓存同时扮演两种角色：

- 对浏览器来说，它是服务器；
- 对源服务器来说，它又是客户端。

2. 基本工作流程

浏览器先把HTTP请求发送给Web缓存。

缓存命中

缓存中已经有该对象：

浏览器 → 缓存 → 直接返回对象

不需要访问源服务器。

缓存未命中

缓存中没有该对象：

1. 缓存向源服务器发送HTTP请求；
2. 源服务器返回对象；

3. 缓存保存对象副本；
4. 缓存把对象返回给浏览器。

3. Web缓存的优点

- 降低用户访问时延；
- 减轻源服务器负担；
- 缓解较慢接入链路造成的拥塞。

4. 缓存对象是否过期

缓存中的对象可能已经不是最新版本，因此HTTP使用**条件GET**进行验证。

请求中可以携带：

If-Modified-Since： 某个时间

如果源服务器中的对象没有修改，则返回：

304 Not Modified

此时服务器不再发送完整对象，缓存直接使用已有副本。

2.3 FTP

FTP是文件传输协议，采用CS体系结构，并使用TCP提供可靠传输。

一、FTP的主要特点

1. 使用两条独立TCP连接

FTP最重要的特点是：

控制信息和文件数据分别通过不同的TCP连接传输。

这称为**带外控制**。

2. 控制连接

控制连接用于传送：

- 用户名和密码；
- 文件操作命令；
- 服务器响应；

服务器控制端口为：21

控制连接通常在整个FTP会话期间一直保持。

3. 数据连接

数据连接用于传输：

- 文件内容；
- 目录列表。

服务器数据端口为：20

每进行一次文件传输，通常建立一条新的数据连接；文件传输结束后，数据连接关闭，但控制连接仍可继续保持。

4. FTP是有状态协议

FTP服务器需要保存当前用户的状态。

2.4 电子邮件：SMTP、POP3、IMAP

一、电子邮件系统组成

电子邮件系统主要包括三部分：

1. 用户代理UA；
2. 邮件服务器；
3. 邮件传输和读取协议。

用户代理

用户编写、发送、阅读和管理邮件的软件。

邮件服务器

保存用户邮箱，并维护待发送邮件队列。

二、电子邮件传输过程

以Alice给Bob发送邮件为例：

1. Alice使用用户代理编写邮件；
2. 邮件提交到Alice的邮件服务器；
3. Alice的邮件服务器使用SMTP连接Bob的邮件服务器；
4. 邮件被存入Bob的邮箱；
5. Bob使用POP3、IMAP或Web方式读取邮件。

三、SMTP

SMTP是简单邮件传输协议。

1. 主要特点

- 使用TCP可靠传输；
- 标准服务器端口为25；
- 采用客户—服务器方式；
- 采用命令—响应交互；
- SMTP客户端主动把邮件推送到服务器，因此属于**推协议**；
- 邮件服务器之间也使用SMTP传输邮件。

2. SMTP三个阶段

1. 建立连接；
2. 邮件传送；
3. 连接释放。

3. SMTP限制与MIME

传统SMTP要求邮件内容使用7位ASCII编码，不适合直接传输中文、图片、音频等非ASCII内容。

MIME对邮件格式进行扩展，使电子邮件能够传输：

- 中文等非ASCII文本；
- 图片、音频、视频；
- 各种附件。

四、POP3

POP3是邮局协议第3版，用于从邮件服务器读取邮件。

标准端口：110

特点

- 使用TCP；
- 邮件通常下载到本地；
- 支持“下载并删除”或“下载并保留”；
- 对服务器端文件夹和邮件状态管理能力较弱。

POP3通常包括：

1. 用户认证阶段；
2. 邮件操作阶段；
3. 更新阶段。

五、IMAP

IMAP是互联网邮件访问协议。

标准端口：143

特点

- 使用TCP；
- 邮件主要保存在服务器上；
- 支持在服务器中建立和管理文件夹；
- 可以只读取邮件的一部分；
- 可以在多台设备之间同步邮件状态；
- 能保持用户在不同会话中的状态。

六、SMTP、POP3、IMAP对比

协议	主要作用	方向	端口	主要特点
SMTP	发送和转发邮件	推送	25	发送方到服务器、服务器到服务器
POP3	下载邮件	拉取	110	简单，主要下载到本地
IMAP	在线访问和管理邮件	拉取	143	邮件保留在服务器，支持文件夹和同步

2.5 DNS

DNS是域名系统，其核心作用是将便于人类记忆的主机名转换为IP地址。

DNS是：

- 分布式数据库；
- 层次化结构；
- 应用层协议；
- 通常使用UDP；
- 端口号为53。

一、DNS的主要作用

1. 主机名到IP地址的转换

这是DNS最基本的功能。

2. 主机别名

一个复杂的规范主机名可以对应一个简单别名。

3. 邮件服务器别名

帮助查找某个域对应的邮件服务器。

4. 负载分配

同一个域名可以对应多个服务器IP地址，从而将请求分配到不同服务器。

二、DNS服务器类型

1. 根域名服务器

根服务器位于DNS层次结构的最高层。

作用：

- 一般不直接保存具体主机的IP地址；
- 知道顶级域名服务器的位置；
- 当无法解析域名时，将查询引导到相应的顶级域名服务器。

2. 顶级域名服务器

TLD服务器负责某个顶级域。

作用：

- 保存或知道相关权威域名服务器的信息；
- 将查询进一步引导到具体域的权威服务器。

3. 权威域名服务器

权威域名服务器保存某个组织或域名的正式DNS记录。

权威服务器可以给出所管理域名的最终、权威答案。

4. 本地域名服务器

也叫默认域名服务器或本地DNS服务器。

特点：

- 通常由ISP、学校或企业提供；
- 是主机发送DNS查询时首先联系的服务器；
- 代表主机继续向其他DNS服务器查询；
- 会缓存以前的查询结果。

三、四类服务器的关系

查询一个域名时，常见过程是：

本地DNS → 根DNS → 顶级域DNS → 权威DNS

各服务器的作用可以记成：

根找顶级，顶级找权威，权威给答案，本地替用户查询并缓存。

四、递归查询

被查询的DNS服务器必须负责完成整个查询过程，并将最终结果返回查询者。

特点：

- 查询者只提出一次请求；
- 被查询服务器承担后续查询工作；
- 对被查询服务器压力较大。

五、迭代查询

被查询服务器如果不知道最终答案，就返回它认为下一步应该联系的DNS服务器地址。

例如：

1. 本地DNS询问根服务器；
2. 根服务器返回顶级域服务器地址；
3. 本地DNS再询问顶级域服务器；
4. 顶级域服务器返回权威服务器地址；
5. 本地DNS询问权威服务器；
6. 权威服务器返回最终IP地址。

特点：

- 每个服务器只返回自己知道的最好信息；
- 查询者负责继续下一步查询；
- DNS层次服务器之间通常采用迭代查询。

实际过程中通常是：

- 主机向本地DNS：递归查询；
- 本地DNS向根、顶级、权威DNS：迭代查询。

七、DNS缓存

本地DNS会保存以前查询得到的结果。

优点：

- 减少查询时间；
- 减少DNS服务器负载；

- 减少网络中的DNS查询流量。

缓存记录不是永久有效的，而是由TTL规定有效时间。

TTL到期后，缓存记录需要重新查询更新。

八、DNS资源记录

DNS是一个分布式数据库，数据库中的基本数据单位称为：

资源记录 Resource Record, RR

每条资源记录通常表示为：

$(Name, Value, Type, TTL)$

其中：

- **Name**：被查询的域名或主机名；
- **Value**：该名称所对应的具体内容；
- **Type**：资源记录类型，决定Name和Value分别表示什么；
- **TTL**：该记录能够在DNS缓存中保留的时间。

需要注意：

Name和Value的含义不是固定的，而是由Type决定的。

1. TTL

TTL是：

Time To Live

表示该资源记录可以在缓存中保存多长时间。

TTL到期后：

- 缓存记录失效；
- 本地DNS需要重新查询；
- 从而避免长期使用已经改变的记录。

这里的DNS TTL与IP首部中的TTL不同：

- DNS中的TTL：缓存有效时间；
- IP中的TTL：数据报允许经过的最大跳数。

2. A记录

A记录用于将**主机名**映射为**IPv4地址**。

此时：

- Name：主机名；
- Value：对应的IPv4地址。

例如：

```
(www.example.com, 192.0.2.10, A, 3600)
```

表示：

```
www.example.com → 192.0.2.10
```

因此，当浏览器需要访问某台Web服务器时，通常查询的就是A记录。

3. NS记录

NS记录用于指出：

哪一台DNS服务器是某个域的权威域名服务器。

此时：

- Name：一个域名；
- Value：负责该域的权威DNS服务器的主机名。

例如：

```
(example.com, dns1.example.com, NS, 86400)
```

表示：

dns1.example.com 是 example.com 域的权威DNS服务器。

4. CNAME记录

CNAME记录用于建立：

别名 → 规范主机名

此时：

- Name：主机的别名；
- Value：主机的规范名称，也称真实名称。

例如：

(www.example.com, server1.example.com, CNAME, 3600)

表示：

www.example.com

只是别名，真正的规范主机名是：

server1.example.com

5. MX记录

MX记录用于指出某个域的**邮件服务器**。

此时：

- Name：邮件域名；
- Value：该域对应的邮件服务器主机名。

例如：

(example.com, mail.example.com, MX, 3600)

表示：

发往 @example.com 的邮件，应发送到 mail.example.com 邮件服务器。

6. 四种资源记录对比

Type	Name表示	Value表示	主要作用
A	主机名	IPv4地址	主机名解析为IP地址
NS	域名	权威DNS服务器主机名	找到负责该域的DNS服务器
CNAME	主机别名	规范主机名	别名解析为真实主机名
MX	邮件域名	邮件服务器主机名	找到该域的邮件服务器

九、DNS查询报文与应答报文

DNS使用客户—服务器方式工作。

- DNS客户端发送查询报文；
- DNS服务器返回应答报文。

DNS查询报文和应答报文采用相同的基本结构：

首部 + 问题区域 + 回答区域 + 权威区域 + 附加信息区域

1. DNS报文首部

DNS报文首部包含以下重要内容。

标识符 Identification

标识符通常为16位。

客户端发送查询时设置一个标识符，服务器在应答中使用相同的标识符。

作用是：

让客户端知道某个应答对应自己之前发送的哪一个查询。

标志位 Flags

标志位用于表示报文的性质和查询要求。

各区域记录数量

首部还会记录：

- 问题区域有多少个查询；
- 回答区域有多少条资源记录；
- 权威区域有多少条资源记录；
- 附加区域有多少条资源记录。

2. 问题区域

问题区域包含客户端想查询的内容，主要包括：

- 查询的名称；
- 查询的记录类型。

3. 回答区域

回答区域包含与问题相匹配的资源记录。

4. 权威区域

权威区域用于提供：

与被查询域有关的权威DNS服务器信息。

5. 附加信息区域

附加信息区域提供与本次查询有关的辅助信息，减少后续查询次数。

第三章 传输层

一、传输层提供的服务

1. 传输层的核心任务

传输层为运行在不同主机上的**应用进程**提供逻辑通信。

层次	提供的通信
网络层	主机到主机
传输层	进程到进程

例如，网络层只能把数据送到你的电脑，而传输层还要判断数据应交给浏览器、微信还是其他应用程序。

2. 发送端和接收端的工作

发送端：

应用层报文 → 分段 → 加传输层首部 → 交给网络层

接收端：

从网络层取得报文段 → 检查首部 → 取出数据 → 交给正确应用进程

3. TCP与UDP

互联网主要有两个传输层协议：

- TCP：面向连接、可靠、有序；
- UDP：无连接、不保证可靠和有序。

两者都提供：

- 多路复用；
- 多路解复用；
- 差错检测。

TCP额外提供：

- 可靠传输；
- 流量控制；
- 拥塞控制；
- 连接管理。

但TCP和UDP都不能保证：

- 固定时延；
- 固定带宽。

二、多路复用与解复用

1. 多路复用

发生在发送端。

一台主机可能同时运行多个应用进程，传输层从不同套接字接收数据，添加源端口、目的端口等首部字段，再交给网络层。

简单理解：

把多个应用进程的数据汇集到网络中发送。

2. 解复用

发生在接收端。

传输层检查报文段首部中的IP地址和端口号，把数据交给正确的套接字和应用进程。

简单理解：

收到网络数据后，判断应当交给哪个应用。

端口号是传输层的服务访问点。通常：

IP地址 + 端口号

可以定位某台主机上的特定应用进程。

3. UDP的解复用

UDP套接字主要通过二元组标识：

(目的IP地址, 目的端口号)

只要两个UDP数据报具有相同的目的IP和目的端口，就会被交给同一个UDP套接字，即使它们的源IP和源端口不同。

例如：

主机A:5000 → 服务器:53
主机B:6000 → 服务器:53

它们都可以交给服务器的53号UDP套接字。

4. TCP的解复用

TCP连接由四元组唯一标识：

(源IP, 源端口, 目的IP, 目的端口)

因此，同一个Web服务器的80端口可以同时支持大量客户端连接。

例如：

A:5000 → B:80
C:6000 → B:80

虽然目的端口都是80，但源IP和源端口不同，所以属于不同TCP连接。

三、UDP

1. UDP的特点

UDP是无连接传输协议。

- 发送前不建立连接；
- 发送端和接收端不保存连接状态；
- 每个UDP数据报独立处理；
- 可能丢失、乱序；
- 不进行流量控制、拥塞控制；
- 首部简单，处理速度快。

2. UDP报文格式

UDP首部只有8字节，由四个16 bit字段组成：

字段	作用
源端口号	标识发送进程
目的端口号	标识接收进程
长度	UDP首部和数据的总字节数
校验和	检测传输中的比特差错

后面是应用层数据。

3. UDP校验和

目的：检测数据传输过程中是否发生比特翻转。

发送端步骤

1. 将校验范围划分为若干个16 bit字；
2. 使用**反码加法**相加；
3. 如果最高位产生进位，将进位回卷到最低位继续相加；
4. 对最终结果逐位取反；
5. 得到校验和。

接收端步骤

接收端把所有16 bit数据字和校验和再次进行反码求和。

若结果为：

1111111111111111

则通过校验；否则说明检测到了差错。

注意：

校验和只能检测差错，不能纠正差错，而且存在某些差错无法被检测，因此属于弱保护机制。

四、可靠数据传输原理

可靠数据传输的目标是：

即使下层信道可能出错或丢包，上层看到的仍像是一条可靠信道。

1. rdt1.0：完全可靠信道

假设下层：

- 不会出现比特差错；
- 不会丢失分组。

发送方直接发送，接收方直接接收，不需要ACK、序号和重传。

这是最理想、最简单的模型。

2. rdt2.0：可能出现比特差错

假设：

- 分组不会丢失；
- 但分组中的比特可能出错。

增加机制：

- 校验和检测差错；

- ACK表示正确接收；
- NAK表示检测到差错；
- 收到NAK后重传。

致命缺陷

ACK或NAK本身也可能损坏。

发送方看到一个损坏的确认报文时，不知道接收方究竟有没有正确收到数据：

- 不重传，可能导致数据丢失；
- 重传，可能产生重复数据。

3. rdt2.1：处理损坏的ACK/NAK

增加序号。

因为是停止等待协议，发送方一次只发送一个未确认分组，因此只需要两个序号：

0, 1

发送方交替发送0号、1号分组。

当ACK或NAK损坏时：

- 发送方重传当前分组；
- 接收方根据序号判断是否为重复分组；
- 重复分组被丢弃，不再交付上层；
- 但接收方仍重新发送确认。

为什么0和1就够

发送方一次只可能等待一个分组的确认，接收方只需判断：

当前到达的是期待的新分组，还是上一个分组的重复副本。

4. rdt2.2：不使用NAK

功能与rdt2.1相同，但只使用ACK。

接收方若收到错误分组，就重新发送对**最后正确接收分组**的ACK。

发送方收到重复ACK后，等价于收到NAK，于是重传当前分组。

例如发送方等待ACK1，却再次收到ACK0，说明1号分组没有被正确接收，需要重传1号分组。

5. rdt3.0：可能出错，也可能丢包

信道现在可能丢失：

- 数据分组；
- ACK确认。

如果分组或ACK丢失，双方可能永久等待，所以引入：

- 倒计时定时器；
- 超时重传。

发送方发送分组后启动定时器：

- 正确ACK按时到达：停止定时器，发送下一个分组；
- 超时仍未收到ACK：重传当前分组，重新启动定时器。

过早超时

分组或ACK可能只是延迟，并没有真正丢失。发送方仍会重传，造成重复分组。

序号机制可以识别并丢弃重复数据，因此不会重复交付给应用层。

五、停止等待与流水线协议

1. 停止等待

发送方每发送一个分组，就必须等待确认后才能发送下一个。

优点是简单，缺点是链路利用率低。

发送方利用率：

$$U_{\text{sender}} = \frac{L/R}{RTT + L/R}$$

其中：

- (L)：分组长度；
- (R)：链路速率；
- (L/R)：发送一个分组的时间；
- RTT：往返时间。

当RTT远大于发送时间时，发送方绝大部分时间都在等待。

2. 流水线协议

允许发送方在没有收到确认前连续发送多个分组。

需要增加：

- 更大的序号空间；
- 发送窗口；

- 发送方缓冲区；
- 接收方可能需要缓冲区；
- 滑动窗口机制。

主要有两种：

- GBN：回退N步；
- SR：选择重传。

六、GBN与SR

1. GBN：回退N步

发送窗口最多允许存在 (N) 个连续未确认分组。

发送方

- 可以连续发送窗口内多个分组；
- 使用累计确认；
- 通常只为最早未确认分组设置一个定时器；
- 最早未确认分组超时后，从该分组开始重传所有未确认分组。

接收方

- 接收窗口大小为1；
- 只接收下一个期望的分组；
- 对乱序分组直接丢弃；
- 重复发送最后一个按序接收分组的ACK。

例子

发送方发送：

0, 1, 2, 3, 4

若2号丢失：

- 接收方正确接收0和1；
- 3和4乱序到达，被丢弃；
- 反复发送ACK1；
- 2号超时后，发送方重传2、3、4。

因此叫“回退N步”。

窗口与序号

若序号字段为 (k) bit，则序号范围为：

$$0 \sim 2^k - 1$$

GBN发送窗口通常要求：

$$N \leq 2^k - 1$$

2. SR：选择重传

SR对每个分组分别确认、分别设置定时器。

发送方

- 每个未确认分组都有自己的定时器；
- 某个分组超时，只重传该分组；
- 收到该分组ACK后单独标记；
- 只有窗口最左端分组确认后，窗口才能向前滑动。

接收方

- 接收窗口大于1；
- 可以接收并缓存乱序分组；
- 对每个正确分组单独发送ACK；
- 等缺失的低序号分组到达后，再按顺序交付上层。

同一个例子

发送：

$$0, 1, 2, 3, 4$$

2号丢失：

- 接收方缓存3、4，并分别发送ACK3、ACK4；
- 2号超时后，只重传2；
- 收到2后，将2、3、4按顺序交付。

3. GBN与SR对比

对比项	GBN	SR
确认方式	累计确认	单独确认
接收窗口	1	大于1
乱序分组	丢弃	缓存
定时器	通常一个	每个未确认分组一个
超时重传	从丢失处开始全部重传	只重传丢失分组
实现复杂度	较低	较高

对比项	GBN	SR
传输效率	较低	较高

SR为了避免新旧分组混淆，发送窗口和接收窗口通常满足：

$$N \leq 2^{k-1}$$

七、TCP基本特性与报文段

1. TCP特性

TCP是：

- 面向连接；
- 点到点；
- 可靠传输；
- 按序交付；
- 全双工；
- 面向字节流；
- 流水线传输；
- 具有发送和接收缓冲区；
- 提供流量控制和拥塞控制。

TCP不是“报文”服务，而是字节流服务。应用层交给TCP的数据被视为连续字节序列。

2. TCP报文段主要字段

字段	作用
源端口、目的端口	多路复用和解复用
序号	当前报文段数据第一个字节的编号
确认号	下一个期望收到的字节编号
首部长度的	TCP首部大小
接收窗口	用于流量控制
校验和	检测差错
SYN	建立连接
FIN	释放连接
RST	复位连接
ACK	确认号字段有效
URG	紧急指针有效
PSH	尽快交付应用

字段	作用
选项	例如MSS等

3. 序号和确认号

TCP序号是按**字节**编号，而不是按报文段编号。

例如，一个TCP报文段：

- 第一个字节序号为1000；
- 携带500字节数据。

则下一个报文段序号为：

$$1000 + 500 = 1500$$

接收方若已正确收到这些字节，就返回：

$$ACK = 1500$$

含义是：

0到1499之前的数据已经收到，下一步期待1500号字节。

TCP通常使用累计确认。

八、TCP超时与可靠传输

1. RTT估计

TCP不能把超时时间设得过短或过长：

- 过短：容易发生不必要重传；
- 过长：真正丢包后恢复太慢。

采样得到：

$$SampleRTT$$

平滑估计：

$$EstimatedRTT = (1 - \alpha)EstimatedRTT + \alpha SampleRTT$$

课件中通常取：

$$\alpha = 0.125$$

RTT偏差：

$$DevRTT = (1 - \beta)DevRTT + \beta |SampleRTT - EstimatedRTT|$$

通常：

$$\beta = 0.25$$

超时时间：

$$TimeoutInterval = EstimatedRTT + 4DevRTT$$

2. TCP可靠传输的主要事件

应用数据到达

- TCP创建报文段；
- 设置序号；
- 发送报文段；
- 若定时器未运行，为最早未确认报文段启动定时器。

超时

- 重传最早未确认报文段；
- 重新启动定时器。

收到ACK

- 更新已确认数据范围；
- 发送窗口向前移动；
- 若仍有未确认数据，重新启动定时器；
- 若所有数据均确认，关闭定时器。

3. 快速重传

TCP不一定等到超时才重传。

如果接收方发现某个报文段缺失，而后续报文段不断到达，会反复返回相同ACK。

发送方收到：

3个重复ACK

后，推测某个报文段已经丢失，立即重传该报文段。

这就是：

快速重传

注意：

“3个重复ACK”通常意味着总共收到了4个相同确认号：1个原始ACK加3个重复ACK。

九、TCP连接管理

1. 三次握手

第一次

客户端发送：

```
SYN=1, seq=x
```

表示请求建立连接。

第二次

服务器发送：

```
SYN=1, ACK=1, seq=y, ack=x+1
```

表示服务器同意连接，同时确认客户端序号。

第三次

客户端发送：

```
ACK=1, ack=y+1
```

连接建立，第三次报文可以携带数据。

2. 四次挥手

TCP是**全双工通信**，客户端和服务端都可以独立发送数据。因此，关闭连接时需要分别关闭两个方向的数据传输。

假设客户端主动关闭连接，客户端当前序号为 (x)，服务器当前序号为 (y)。

第一次

客户端发送：

```
FIN=1, ACK=1, seq=x, ack=y
```

表示客户端已经没有数据需要发送，请求关闭“客户端到服务器”方向的连接。

发送后，客户端进入 `FIN_WAIT_1` 状态。

需要注意：

- FIN表示客户端以后不再发送数据；
- 客户端仍然可以接收服务器发送的数据；

- FIN会占用一个序号，因此客户端下一个序号变为 $x+1$ 。

第二次

服务器收到FIN后发送：

```
ACK=1, seq=y, ack=x+1
```

表示服务器已经收到客户端的关闭请求。

发送后：

- 服务器进入 `CLOSE_WAIT` 状态；
- 客户端收到ACK后进入 `FIN_WAIT_2` 状态。

此时只是关闭了客户端向服务器发送数据的方向，服务器仍然可以继续向客户端发送尚未传输完的数据，因此TCP连接处于**半关闭状态**。

第二次报文是纯ACK，不携带数据时不占用序号，所以服务器的序号仍然可以是 y 。

第三次

服务器处理完剩余数据后发送：

```
FIN=1, ACK=1, seq=y, ack=x+1
```

表示服务器的数据也已经发送完毕，请求关闭“服务器到客户端”方向的连接。

发送后，服务器进入 `LAST_ACK` 状态。

服务器的FIN同样会占用一个序号，因此客户端下一次期待服务器发送的序号为：

```
y+1
```

如果服务器在第二次和第三次挥手之间又发送了若干字节的数据，那么第三次挥手中的 `seq` 不一定仍然是 y ，而应当等于服务器发送完这些数据后的下一个序号。

第四次

客户端收到服务器的FIN后发送：

```
ACK=1, seq=x+1, ack=y+1
```

表示客户端已经收到服务器的关闭请求。

发送后：

- 客户端进入 `TIME_WAIT` 状态；
- 服务器收到该ACK后进入 `CLOSED` 状态；
- 客户端等待 `2MSL` 后进入 `CLOSED` 状态。

至此，TCP连接完全关闭。

十、TCP流量控制

流量控制解决的是：

防止发送方速度过快，压垮接收方缓冲区。

接收方在TCP首部的接收窗口字段中通告：

$rwnd$

剩余接收缓冲空间可表示为：

$$rwnd = RcvBuffer - (LastByteRcvd - LastByteRead)$$

发送方需要保证未确认数据量不超过接收窗口：

$$LastByteSent - LastByteAcked \leq rwnd$$

接收方应用读取数据后，缓冲空间增大，通告的 $rwnd$ 也会增大。

十一、拥塞控制原理

拥塞是指：

太多发送方以过快速度向网络发送过多数据，超过了网络处理能力。

可能导致：

- 路由器队列增长；
- 排队时延增大；
- 缓冲区溢出；
- 分组丢失；
- 发送方重传；
- 重传进一步加剧拥塞；
- 已经消耗上游链路资源的分组最终被丢弃。

两种拥塞控制方式

网络辅助的拥塞控制

路由器显式地向端系统提供拥塞信息。

例如：

- 标记报文；
- 通知发送方降低速率。

端到端拥塞控制

网络层不直接提供明确反馈，发送端根据：

- 丢包；
- 超时；
- 重复ACK；
- RTT变化

推测网络发生拥塞。

传统TCP主要采用端到端拥塞控制。

十二、TCP拥塞控制

1. 两个核心变量

拥塞窗口

cwnd

表示网络当前允许发送方拥有的未确认数据量。

慢启动阈值

ssthresh

决定TCP何时从慢启动进入拥塞避免。

实际允许的发送窗口为：

$$\min(cwnd, rwnd)$$

因此：

- rwnd限制接收方能力；
- cwnd限制网络承载能力。

2. 慢启动

连接开始时，TCP不知道网络能承受多大流量，因此从较小cwnd开始。

通常：

$$cwnd = 1MSS$$

每收到一个新的ACK：

$$cwnd = cwnd + 1MSS$$

因此每经过一个RTT，cwnd大约翻倍：

$$1, 2, 4, 8, 16, \dots$$

虽然叫“慢启动”，但实际上属于指数增长。

当：

$$cwnd \geq ssthresh$$

进入拥塞避免阶段。

3. 拥塞避免

拥塞避免采用加性增大：

每经过一个RTT，cwnd大约增加1 MSS。

变化近似为：

$$cwnd \leftarrow cwnd + \frac{MSS^2}{cwnd}$$

每收到一个ACK增加一点，一整个RTT合计约增加1 MSS。

因此窗口呈线性增长。

4. AIMD

TCP总体采用：

加性增大，乘性减小

即AIMD：

- Additive Increase：未发现拥塞时，窗口逐渐线性增加；
- Multiplicative Decrease：发现拥塞时，窗口通常减半。

窗口图形通常呈锯齿状。

5. 超时后的处理

超时被认为是较严重的拥塞。

假设丢包时窗口为 (cwnd)：

$$ssthresh = \frac{cwnd}{2}$$

然后：

$$cwnd = 1MSS$$

重新进入慢启动。

变化过程：

窗口减到1
→ 指数增长
→ 到达ssthresh
→ 线性增长

6. 三个重复ACK后的处理

三个重复ACK说明：

- 某个报文段丢失；
- 但后续数据还能到达；
- 网络仍具有一定传输能力。

因此不必像超时那样把窗口降到1。

TCP执行：

1. 快速重传；
2. 设置：

$$ssthresh = \frac{cwnd}{2}$$

3. 进入快速恢复；
4. 收到新的有效ACK后，令cwnd回到ssthresh附近；
5. 进入拥塞避免。

TCP Reno快速恢复过程中常设置：

$$cwnd = ssthresh + 3MSS$$

每收到一个额外重复ACK，再增加1 MSS；收到新ACK后：

$$cwnd = ssthresh$$

十三、流量控制与拥塞控制辨析

对比项	流量控制	拥塞控制
保护对象	接收方	整个网络
原因	接收缓冲区可能满	网络负载超过承载能力
控制变量	rwnd	cwnd
信息来源	接收方通告	丢包、超时、重复ACK等
目标	不压垮接收端	不压垮网络

二者联合控制：

$$\text{发送窗口} = \min(rwnd, cwnd)$$

第四章 网络层

一、网络层概述

1. 网络层的基本作用

网络层负责将数据从发送主机传送到接收主机。

发送端：

传输层报文段 → 封装成IP数据报 → 交给链路层

中间路由器：

读取IP首部中的目的IP地址 → 查询转发表 → 选择输出接口

接收端：

收到IP数据报 → 取出传输层报文段 → 交给TCP或UDP

网络层协议存在于：

- 每台主机；
- 每台路由器。

路由器主要处理到网络层，一般不处理普通用户数据的传输层和应用层内容。

2. 网络层的两个核心功能

转发 Forwarding

将到达路由器输入端口的数据报，移动到适当的输出端口。

特点：

- 是路由器内部的局部操作；
- 针对一个到达的数据报；
- 根据转发表快速完成；
- 属于数据平面的功能；
- 时间尺度通常很短。

路由 Routing

确定数据报从源主机到目的主机经过的完整路径。

特点：

- 是全局路径计算；
- 使用路由算法；
- 计算结果用于生成转发表；
- 属于控制平面的功能。

二者关系

路由算法生成转发表，转发过程使用转发表

3. 网络层服务模型

互联网网络层提供的是：

尽力而为服务 Best Effort

网络层会尽力传输数据报，但不保证：

- 数据一定到达；
- 数据按顺序到达；
- 数据只到达一次；
- 固定传输时延；
- 固定可用带宽。

因此，可靠性主要由端系统的TCP等协议实现。

二、虚电路网络与数据报网络

1. 虚电路网络

虚电路网络提供面向连接的网络层服务。

通信前先建立虚电路：

- 建立连接
- 沿途路由器保存连接状态
 - 传输数据
 - 拆除连接

每个分组携带的是**虚电路标识VCI**，而不是完整目的地址。

沿途路由器维护类似下面的虚电路表：

输入接口	输入VC号	输出接口	输出VC号
1	12	3	27

虚电路号只具有链路局部意义，因此经过路由器时可以改变。

特点

- 发送数据前需要建立连接；
- 同一虚电路上的分组通常沿相同路径；
- 路由器需要维护连接状态；
- 可以为虚电路分配带宽和缓冲区；

2. 数据报网络

互联网采用数据报网络。

特点：

- 网络层不建立连接；
- 路由器不维护端到端连接状态；
- 每个分组携带完整目的IP地址；
- 每个数据报被独立处理；
- 同一数据流中的不同数据报可能走不同路径；
- 网络核心相对简单，复杂功能主要在端系统实现。

3. 虚电路与数据报网络对比

对比项	虚电路网络	数据报网络
网络层服务	面向连接	无连接
传输前建立连接	需要	不需要
分组携带内容	VC标识	完整目的IP
路由器保存连接状态	保存	不保存
分组路径	通常相同	可能不同
故障恢复	路径故障可能使连接失效	可重新计算路径
核心网络	较复杂	较简单
典型代表	ATM、X.25	Internet

三、数据报转发表与最长前缀匹配

1. 为什么不直接存储每个IP地址

IPv4共有：

$$2^{32}$$

个可能地址，路由器不可能为每个IP地址单独保存一条记录。

因此转发表保存的是：

目的网络前缀 → 输出接口或下一跳

2. 最长前缀匹配

当一个目的IP地址同时匹配多条转发表项目时，选择匹配前缀最长的一项。

例如转发表：

目的网络	输出接口
200.23.16.0/20	0
200.23.18.0/23	1
200.23.18.128/25	2
默认	3

如果目的地址为：

200.23.18.150

它可能同时匹配：

- /20；
- /23；
- /25。

应选择最长的：

/25

因此从接口2转发。

四、路由器内部结构

路由器主要由四部分组成：

1. 输入端口；
2. 交换结构；
3. 输出端口；
4. 路由选择处理器。

1. 输入端口

输入端口依次完成：

- 物理层接收比特；
- 链路层解封装；
- 检查IP首部；
- 查询转发表；
- 确定输出接口；
- 必要时在输入端口排队。

输入排队与HOL阻塞

如果多个输入端口的数据报争用同一个输出端口，就可能发生输入排队。

队首阻塞HOL：

队首数据报暂时无法转发，导致它后面本来可以转发的数据报也被阻塞。

2. 交换结构

交换结构负责把分组从输入端口送到输出端口。

经内存交换

过程：

输入端口→内存→输出端口

特点：

- 由CPU或内存控制；
- 每个分组需要先写入内存，再从内存读出；
- 速度受内存带宽限制；
- 通常一次只能处理一个分组。

经总线交换

所有端口共享一条总线。

特点：

- 输入端口直接把分组放到共享总线上；
- 输出端口根据标签接收；
- 一次通常只能有一个分组使用总线；
- 性能受总线带宽限制。

经互联网络交换

例如纵横交换矩阵Crossbar。

特点：

- 可以让多个输入—输出对同时传输；
- 并行能力较强；
- 性能优于单内存和单总线方式；
- 若多个输入同时争用同一输出，仍会发生冲突。

3. 输出端口

输出端口主要完成：

- 输出队列管理；
- 调度；
- 链路层封装；
- 物理层发送。

当分组到达输出端口的速度大于输出链路发送速度时，就会发生排队。

如果缓冲区已满：

后续到达的数据报会被丢弃

这就是路由器丢包的主要原因之一。

4. 路由处理器

路由处理器位于控制平面，主要负责：

- 运行RIP、OSPF、BGP等路由协议；
- 计算路由；
- 生成和维护转发表；
- 管理路由器。

五、IPv4数据报格式

IPv4数据报由：

首部 + 数据

组成。

IPv4标准首部长度为：

20字节

有选项时首部可以更长。

IPv4数据报最大总长度为：

$$2^{16} - 1 = 65535 \text{ 字节}$$

1. IPv4首部主要字段

字段	含义
版本	IPv4中为4
首部长度的IHL	首部长度的单位是4字节
服务类型	区分服务、拥塞通知等
总长度	首部和数据的总字节数
标识	标识属于同一个原始数据报的分片
标志	DF、MF等分片控制信息
片偏移	当前分片在原数据报中的位置
TTL	剩余最大跳数
协议	指明上层协议
首部校验和	检查IPv4首部是否出错
源IP地址	发送方IP地址
目的IP地址	接收方IP地址
选项	可选字段

2. 首部长度的IHL

IHL的单位不是字节，而是：

4字节

例如：

$$IHL = 5$$

则首部长度的为：

$$5 \times 4 = 20 \text{ 字节}$$

3. TTL

TTL表示数据报允许经过的剩余最大跳数。

每经过一台路由器：

$$TTL = TTL - 1$$

若TTL减为0：

- 路由器丢弃数据报；
- 向源主机发送ICMP超时报文。

作用：

防止由于路由环路，数据报永久在网络中循环。

4. 协议字段

协议字段说明IP数据部分交给哪个上层协议。

常见值：

值	协议
1	ICMP
6	TCP
17	UDP

六、IPv4分片与重组

1. 为什么需要分片

不同链路具有不同的MTU。

MTU是：

链路层一帧能够承载的最大IP数据报长度

如果一个IPv4数据报大于下一条链路的MTU，就可能被分成若干个较小的数据报。

IPv4分片只在最终目的主机进行重组，中间路由器一般不重组。

2. 分片相关字段

标识Identification

同一个原始数据报产生的所有分片具有相同标识。

MF标志

- MF=1：后面还有分片；
- MF=0：这是最后一个分片。

片偏移

表示当前分片的数据，在原始数据中的起始位置。

单位是：

8字节

因此片偏移不是直接写字节数，而是：

$$\text{片偏移} = \frac{\text{当前分片前已有数据字节数}}{8}$$

3. 分片通用公式

设：

- 原始总长度为 (L)；
- IP首部长度为 (H)；
- MTU为 (M)。

每个非最后分片最多携带的数据长度：

$$D = \left\lfloor \frac{M - H}{8} \right\rfloor \times 8$$

之所以要乘8，是为了使后续分片的片偏移为整数。

分片数：

$$\left\lceil \frac{L - H}{D} \right\rceil$$

第 (i) 个分片的偏移量：

$$\frac{\text{前面所有分片的数据长度之和}}{8}$$

4. 课件例题

原始IPv4数据报：

- 总长度4000字节；
- 首部20字节；
- 数据3980字节；
- MTU=1500字节。

每个完整分片最大数据：

$$1500 - 20 = 1480 \text{ 字节}$$

1480能被8整除。

第一片

- 数据：1480；
- 总长度：1500；
- 偏移：0；

- MF=1。

第二片

- 数据：1480；
- 总长度：1500；
- 偏移：

$$1480/8 = 185$$

- MF=1。

第三片

剩余数据：

$$3980 - 1480 - 1480 = 1020$$

因此：

- 数据：1020；
- 总长度：

$$1020 + 20 = 1040$$

- 偏移：

$$(1480 + 1480)/8 = 370$$

- MF=0。

结果：

分片	总长度	偏移	MF
1	1500	0	1
2	1500	185	1
3	1040	370	0

七、IPv4编址与子网

1. IPv4地址

IPv4地址长度为：

32 bit

通常写成点分十进制：

192.168.1.10

每8 bit转换为一个0~255的十进制数。

IP地址实际分配给的是：

网络接口

不是笼统地分配给整台设备。

因此：

- 普通主机通常有一个活动接口；
- 路由器有多个接口，因此一般有多个IP地址。

2. 网络前缀与主机号

CIDR形式：

a.b.c.d/n

其中：

- 前 (n) 位是网络前缀；
- 后 (32-n) 位是主机号。

地址总数：

$$2^{32-n}$$

传统IPv4子网中可分配给主机的地址数通常为：

$$2^{32-n} - 2$$

减去：

- 网络地址；
- 广播地址。

3. 子网掩码

子网掩码：

- 网络位全部为1；
- 主机位全部为0。

例如：

```
/24 = 255.255.255.0
/25 = 255.255.255.128
/26 = 255.255.255.192
/27 = 255.255.255.224
/28 = 255.255.255.240
```

网络地址计算：

网络地址IP地址 AND 子网掩码

4. 网络地址计算例题

已知：

```
128.14.35.7/20
```

/20掩码为：

```
255.255.240.0
```

第三个字节：

```
35 = 00100011
240 = 11110000
AND = 00100000 = 32
```

因此网络地址为：

128.14.32.0

地址范围：

```
128.14.32.0~128.14.47.255
```

广播地址：

```
128.14.47.255
```

5. 判断两台主机是否在同一子网

分别计算：

$IP_1 \text{ AND } Mask$

和：

结果相同，则在同一子网；不同，则需要经过路由器通信。

八、子网划分

1. 等长子网划分

假设原网络前缀长度为 (n)，从主机位中借 (s) 位作为子网位。

新前缀长度：

$$n + s$$

子网数量：

$$2^s$$

每个子网地址总数：

$$2^{32-n-s}$$

每个子网传统可用主机数：

$$2^{32-n-s} - 2$$

2. 子网划分基本步骤

1. 根据所需子网数或主机数确定借位数；
2. 求出新前缀长度；
3. 写出新子网掩码；
4. 计算块大小；
5. 依次确定各子网网络地址；
6. 确定广播地址；
7. 确定可用主机范围。

3. 块大小法

观察子网掩码中第一个不等于255的字节。

块大小：

$$256 - \text{该字节的掩码值}$$

例如：

255.255.255.224

块大小：

$$256 - 224 = 32$$

因此子网网络地址依次为：

```
192.168.0.0
192.168.0.32
192.168.0.64
192.168.0.96
192.168.0.128
192.168.0.160
192.168.0.192
192.168.0.224
```

以192.168.0.64/27为例：

- 网络地址：192.168.0.64；
- 广播地址：192.168.0.95；
- 可用地址：192.168.0.65～192.168.0.94；
- 可用主机数：

$$2^5 - 2 = 30$$

4. 按主机数量选择前缀

若每个子网需要 (H) 台主机，应找到最小主机位数 (h)，使：

$$2^h - 2 \geq H$$

新前缀长度为：

$$32 - h$$

例如每个子网需要15台主机：

- $(2^4 - 2 = 14)$ ，不够；
- $(2^5 - 2 = 30)$ ，够。

因此保留5位主机位：

$$32 - 5 = 27$$

应采用：

```
/27
```

九、路由聚合

路由聚合把多个连续网络合并成一个较短的公共前缀。

例如：

```
200.23.16.0/23
200.23.18.0/23
200.23.20.0/23
200.23.22.0/23
```

如果它们拥有共同的高位前缀，可以聚合为：

```
200.23.16.0/21
```

作用：

- 减少路由表条目；
- 降低路由更新开销；
- 提高路由查找效率；
- 提高互联网路由系统的可扩展性。

聚合计算方法

1. 将网络地址转换为二进制；
2. 从最高位开始比较；
3. 找出连续相同的公共前缀；
4. 公共前缀位数就是聚合后的前缀长度；
5. 公共前缀后的位全部补0。

十、DHCP

DHCP用于让主机自动获取网络配置。

DHCP不仅分配IP地址，还可以提供：

- 子网掩码；
- 默认网关；
- DNS服务器地址；

DHCP使用UDP：

- 服务器端口67；
- 客户端端口68。

DHCP四步过程

记忆为：

DORA

DHCP Discover

客户端刚接入网络，没有IP地址，因此发送广播：

网络中有没有DHCP服务器？

DHCP Offer

DHCP服务器返回可用地址和相关配置。

DHCP Request

客户端选择一个服务器的方案，广播请求使用该地址。

DHCP ACK

服务器确认分配，客户端开始使用该IP地址。

DHCP采用租约机制，租期到期前客户端可以续租。

十一、NAT

NAT用于在私有地址与公网地址之间进行转换。

1. NAT基本思想

多个内网设备可以共享一个公网IP地址，通过不同端口号区分不同连接。

NAT转换表可能为：

内网地址和端口	公网地址和端口
10.0.0.1:3345	138.76.29.7:5001
10.0.0.2:2000	138.76.29.7:5002

2. 内网向外发送

原分组：

源IP=10.0.0.1
源端口=3345

经过NAT后改为：

源IP=138.76.29.7
源端口=5001

同时在NAT转换表中记录映射。

3. 外部响应返回

外部服务器把数据发送到：

```
138.76.29.7:5001
```

NAT查询转换表，将目的地址和端口改为：

```
10.0.0.1:3345
```

再转发给对应内网主机。

4. NAT优点与问题

优点：

- 节省公网IPv4地址；
- 内网地址可以自由规划；
- 更换ISP时不必修改所有内网地址；
- 外部不能直接看到内部地址结构。

问题：

- 修改IP地址和端口，破坏端到端原则；
- 对P2P、主动连接、部分应用协议造成困难；
- 路由器承担了部分传输层功能；

十二、ICMP

ICMP用于：

- 报告网络层错误；
- 传递网络控制和诊断信息。

ICMP报文被封装在IP数据报中，但在逻辑上属于网络层协议。

1. ping

ping使用：

- ICMP Echo Request；
- ICMP Echo Reply。

作用：

- 检查目的主机是否可达；

- 测量往返时延RTT；
- 观察分组丢失情况。

ping不使用TCP或UDP端口号。

2. traceroute或tracert

基本原理是逐步增加TTL。

第一批探测分组：

$$TTL = 1$$

第一个路由器将TTL减到0，返回ICMP Time Exceeded。

第二批：

$$TTL = 2$$

第二个路由器返回ICMP超时。

依次增加TTL，就可以得到沿途路由器地址。

核心过程：

```
TTL=1 → 第1跳返回ICMP
TTL=2 → 第2跳返回ICMP
TTL=3 → 第3跳返回ICMP
.....
```

不同操作系统的探测报文实现可能不同，但共同原理都是利用TTL和ICMP超时报文。

十三、IPv6

1. IPv6产生原因

IPv4面临：

- 32位地址空间不足；
- 路由表规模增大；
- 首部处理较复杂；
- NAT破坏端到端通信。

IPv6地址长度为：

128 bit

理论地址数量为：

$$2^{128}$$

2. IPv6表示方法

IPv6使用8组十六进制数，每组16 bit，例如：

```
2001:0db8:0000:0000:0000:ff00:0042:8329
```

前导0可以省略：

```
2001:db8:0:0:0:ff00:42:8329
```

连续的全0组可以用一次 :: 压缩：

```
2001:db8::ff00:42:8329
```

一个地址中 :: 只能使用一次。

3. IPv6基本首部

IPv6基本首部固定为：

40字节

与IPv4相比：

- 没有首部校验和；
- 基本首部中没有分片字段；
- 不允许中间路由器进行分片；
- 选项通过扩展首部实现；
- 处理效率更高。

IPv6的“跳数限制”相当于IPv4的TTL。

4. IPv4向IPv6过渡

双协议栈

设备同时运行IPv4和IPv6，根据通信对象选择相应协议。

隧道技术

当两个IPv6网络之间经过IPv4网络时，把IPv6数据报封装进IPv4数据报中传输。

十四、路由算法基本概念

网络可抽象成图：

$$G = (N, E)$$

其中：

- (N)：路由器节点集合；
- (E)：链路集合；
- (c(x,y))：链路 (x -> y) 的代价。

一条路径的总代价为各链路代价之和：

$$c(x_1, x_2, \dots, x_k) = \sum_{i=1}^{k-1} c(x_i, x_{i+1})$$

路由算法的目标通常是找到最小代价路径。

十五、链路状态算法LS

链路状态算法要求每个路由器知道：

- 完整网络拓扑；
- 所有链路代价。

通过链路状态广播，使每台路由器获得相同的拓扑数据库，然后独立运行Dijkstra算法。

1. Dijkstra符号

设源节点为 (u)。

- (N')：已经确定最短路径的节点集合；
- (D(v))：当前从 (u) 到 (v) 的最小代价估计；
- (p(v))：当前最短路径上 (v) 的前驱节点；
- (c(x,y))：节点 (x) 到 (y) 的直接链路代价。

2. Dijkstra算法步骤

初始化

$$N' = u$$

对于与 (u) 直接相邻的节点：

$$D(v) = c(u, v)$$

否则：

$$D(v) = \infty$$

每轮操作

1. 在不属于 (N') 的节点中，选择 (D(w)) 最小的节点 (w)；
2. 将 (w) 加入 (N')；

3. 更新 (w) 的邻居：

$$D(v) = \min D(v), D(w) + c(w, v)$$

4. 若通过 (w) 的路径更短，则：

$$p(v) = w$$

5. 重复直到所有节点加入 (N')。

易错点

Dijkstra计算的是源节点到所有节点的最短路径，但转发表中填写的是：

从源节点出发的第一跳

例如最短路径：

$u \rightarrow x \rightarrow y \rightarrow z$

则从u到z的下一跳是x，不是y或z。

十六、距离向量算法DV

距离向量算法基于Bellman-Ford方程：

$$d_x(y) = \min_{v \in N(x)} c(x, v) + d_v(y)$$

含义：

节点x到目的y的最短路径，一定先经过x的某个邻居v。

其中：

- $c(x, v)$ ：x到邻居v的直接链路代价；
- $d_v(y)$ ：邻居v到目的y的最短距离估计。

1. DV工作过程

每个节点只知道：

- 到相邻节点的链路代价；
- 邻居发送给自己的距离向量。

过程：

1. 初始化本节点距离向量；
2. 将距离向量发送给邻居；

- 3. 收到邻居距离向量；
- 4. 使用Bellman-Ford公式重新计算；
- 5. 若结果发生变化，通知邻居；
- 6. 反复迭代直到收敛。

特点：

- 分布式；
- 迭代式；
- 异步；
- 每个节点只与邻居交换信息。

2. 无穷计数问题

当链路代价突然增大或链路失效时，两个相邻路由器可能互相误认为对方仍有可达路径。

结果：

2→3→4→5→.....

路由代价逐步增大，收敛很慢，这称为：

无穷计数问题

常见缓解方法：

- 毒性逆转Poisoned Reverse；
- 水平分割；
- 触发更新；
- 设定较小的“无穷大”值。

毒性逆转：

如果x到目的z的路径经过邻居y，那么x向y通告到z的距离为无穷大。

但毒性逆转不能解决所有多节点路由环路。

3. LS与DV对比

对比项	LS	DV
掌握信息	完整拓扑和链路代价	只知道邻居信息
典型算法	Dijkstra	Bellman-Ford
信息传播	链路状态洪泛	与邻居交换距离向量
计算位置	每个路由器独立计算全图	分布式迭代计算

对比项	LS	DV
收敛速度	通常较快	可能较慢
路由环路	较少	可能出现
消息规模	洪泛开销较大	邻居间周期交换
典型协议	OSPF	RIP

十七、分层路由与自治系统

互联网规模非常大，如果每台路由器都保存全互联网详细拓扑，会造成：

- 路由表过大；
- 路由更新过多；
- 算法难以扩展；
- 不同组织无法独立管理网络。

因此把互联网划分为多个：

自治系统AS

每个AS有唯一的AS号ASN，并由一个组织独立管理。

1. AS内部路由

AS内部使用内部网关协议IGP，例如：

- RIP；
- OSPF；
- IS-IS。

2. AS之间路由

不同AS之间使用外部网关协议：

- BGP。

分层路由解决两个主要问题：

- 规模可扩展性；
- 管理自治性。

十八、RIP

RIP是基于距离向量算法的内部网关协议。

主要特点：

- 路径代价使用跳数；
- 每经过一台路由器，跳数加1；
- 最大有效跳数为15；
- 16表示不可达；
- 使用UDP；
- 端口号520；
- 周期性向邻居发送路由表信息。

优点

- 实现简单；
- 配置容易；
- 适合小型网络。

缺点

- 最大只能15跳；
- 收敛速度较慢；
- 可能出现路由环路；
- 存在无穷计数问题；
- 只看跳数，不能反映带宽和时延。

十九、OSPF

OSPF是基于链路状态算法的内部网关协议。

主要特点：

- 每台路由器测量自己的链路状态和代价；
- 生成链路状态通告LSA；
- 将LSA洪泛到整个区域；
- 每台路由器建立相同的链路状态数据库；
- 使用Dijkstra算法计算最短路径；
- 直接封装在IP中，协议号89；
- 支持认证；
- 支持分层区域设计。

OSPF代价可以根据链路带宽等指标设置，不只是跳数。

OSPF层次

- 一个AS可以划分为多个区域；
- 存在骨干区域Area 0；
- 区域内部保存较详细路由；

- 区域之间由区域边界路由器交换信息。

二十、BGP

BGP用于自治系统之间的路由选择，是路径向量协议。

主要特点：

- eBGP：不同AS之间交换路由；
- iBGP：同一AS内部传播外部路由信息；
- 使用TCP连接；
- 端口号179；
- 路由信息包含经过的AS路径；
- 路由选择受到策略影响，不一定选择物理最短路径。

重要属性包括：

- AS-PATH；
- NEXT-HOP；
- LOCAL-PREF；
- MED。

AS-PATH

记录一条路由经过的AS序列。

作用：

- 防止AS级路由环路；
- 可作为路由选择依据。

如果某AS收到的AS-PATH中已经包含自己的AS号，就拒绝该路由。

BGP的核心

RIP和OSPF主要考虑“哪条路径代价更小”，BGP还要考虑商业关系、管理策略和安全政策。

二十一、RIP、OSPF与BGP对比

对比项	RIP	OSPF	BGP
使用范围	AS内部	AS内部	AS之间
类型	距离向量	链路状态	路径向量
主要算法	Bellman-Ford	Dijkstra	策略和路径属性
度量	跳数	链路代价	AS路径及策略

对比项	RIP	OSPF	BGP
最大跳数	15	无15跳限制	不按路由器跳数限制
传输方式	UDP 520	直接使用IP，协议89	TCP 179
拓扑信息	邻居距离向量	完整区域拓扑	AS路径信息
收敛速度	较慢	较快	受策略影响
适用网络	小型网络	中大型AS内部	全球互联网AS间

第五章 链路层和以太网

一、链路层基本概念

1. 节点、链路和帧

运行链路层协议的设备称为**节点**，包括：

- 主机；
- 路由器；
- 交换机；

连接相邻节点的通信信道称为**链路**，可以是：

- 有线链路；
- 无线链路；
- 局域网共享链路；
- 点到点链路。

链路层的数据单位称为：

帧 Frame

2. 链路层传输范围

链路层负责的是：

相邻节点之间的传输

例如：

主机A → 路由器R1 → 路由器R2 → 主机B

整个通信过程中可能经过三条链路，每条链路可以采用不同协议：

- A到R1：Wi-Fi；
- R1到R2：以太网；

- R2到B：其他局域网协议。

因此，同一个IP数据报在端到端传输过程中，可能被多次封装成不同的链路层帧。

二、链路层提供的服务

1. 成帧

链路层把网络层数据报封装进帧中：

帧首部 + IP数据报 + 帧尾部

帧首部可能包含：

- 源MAC地址；
- 目的MAC地址；
- 类型字段；
- 控制信息。

帧尾部通常包含：

- CRC/FCS差错检测码。

2. 链路接入

当多台设备共享一条广播信道时，链路层必须决定：

哪个节点在什么时间可以使用信道？

这由介质访问控制协议MAC协议完成。

注意：

- MAC既可以指介质访问控制协议；
- 也常用于“MAC地址”这一术语。

3. 可靠交付

部分链路层协议可以实现相邻节点间的可靠传输，例如通过：

- 确认；
- 超时；
- 重传。

无线链路误码率较高，因此更可能在链路层进行重传；光纤、双绞线等低误码链路中，链路层可靠重传可能不常使用。

4. 流量控制

链路层可以协调相邻发送方与接收方的速度，防止发送方过快，使接收方来不及处理。

5. 差错检测与纠正

链路传输过程中，数据可能受到各种影响，从而出现比特错误。

链路层可采用：

- 奇偶校验；
- 校验和；
- CRC；

检测错误。

检测出错误后，可能：

- 丢弃帧；
- 通知发送方重传；
- 在具备纠错能力时直接纠正。

6. 半双工与全双工

半双工

双方都可以发送，但同一时刻只能有一个方向传输。

全双工

双方可以同时发送和接收。

现代交换式以太网通常采用全双工，不需要使用CSMA/CD。

三、链路层在哪里实现

链路层通常在**网络适配器NIC**中实现，也就是网卡。

网卡同时实现：

- 链路层功能；
- 部分物理层功能。

网卡通常是：

- 硬件；
- 固件；
- 驱动程序；

的综合体。

发送适配器主要负责：

1. 接收网络层数据报；
2. 封装成帧；
3. 加入差错检测码；
4. 通过物理链路发送。

接收适配器主要负责：

1. 接收帧；
2. 检查是否出错；
3. 检查目的MAC；
4. 解封装；
5. 将IP数据报交给网络层。

四、差错检测基本原理

发送端将数据 (D) 与差错检测位EDC一起发送：

$$D + EDC$$

接收端根据收到的 (D') 和 (EDC') 进行检查。

需要注意：

差错检测并不是100%可靠，某些错误模式可能恰好不能被检测出来。

通常：

- 冗余位越多；
- 算法越强；

漏检概率越低。

五、奇偶校验

1. 单比特奇偶校验

偶校验

使数据和校验位中“1”的总数为偶数。

例如数据中已有奇数个1，则校验位为1；已有偶数个1，则校验位为0。

奇校验

使数据和校验位中“1”的总数为奇数。

2. 检错能力

单比特奇偶校验可以检测：

- 所有奇数个比特错误；
- 特别是单比特错误。

但如果发生偶数个比特错误，可能检测不出来。

3. 二维奇偶校验

将数据排列成矩阵，分别设置：

- 行校验位；
- 列校验位。

若只有一个比特出错：

- 某一行校验失败；
- 某一系列校验失败；

行列交点就是错误位置，因此可以：

检测并纠正单比特错误

六、Internet校验和

1. 发送端

1. 将数据划分成若干个16 bit字；
2. 使用反码加法相加；
3. 对结果按位取反；
4. 将结果放入校验和字段。

2. 接收端

接收端将：

- 所有16 bit数据字；
- 校验和字段；

进行反码加法。

若结果为全1：

- 未检测到错误；
- 但不代表绝对没有错误。

若结果不是全1：

- 检测到错误。

七、CRC循环冗余校验

1. 基本符号

设：

- 原始数据为 (D)；
- 生成多项式对应的二进制除数为 (G)；
- (G) 的最高次数为 (r)；
- CRC余数为 (R)。

如果生成多项式最高次数为 (r)，那么：

- (G) 有 (r+1) 位；
- CRC余数 (R) 有 (r) 位。

例如：

$$G(x) = x^3 + x^2 + 1$$

对应二进制：

$$G = 1101$$

最高次数为3，因此：

$$r = 3$$

CRC余数为3位。

2. 模2运算

CRC使用模2运算：

- 加法不进位；
- 减法不借位；
- 加法和减法都等价于异或XOR。

规则：

$$0 \oplus 0 = 0$$

$$0 \oplus 1 = 1$$

$$1 \oplus 0 = 1$$

$$1 \oplus 1 = 0$$

因此CRC长除法中使用异或，不使用普通减法。

3. 发送端计算过程

设数据为 (D)，生成多项式最高次数为 (r)。

第一步：在数据后补 (r) 个0

$$D \times 2^r$$

二进制中相当于在D后面补 (r) 个0。

第二步：模2除以G

$$R = \text{remainder} \left(\frac{D \times 2^r}{G} \right)$$

第三步：发送数据

将余数R放到原数据后面：

$$T = D \parallel R$$

其中 (T) 就是实际发送的比特串。

它满足：

$$T \div G \text{ 的余数为 } 0$$

5. 接收端如何判断错误

接收方收到比特串 (T')，用相同生成多项式G进行模2除法：

$$T' \div G$$

- 余数为0：未检测到错误；
- 余数非0：检测到错误。

注意：

余数为0只表示“没有检测到错误”，并不能保证绝对没有错误。

7. CRC检错性能

在相应生成多项式条件下，CRC可以：

- 检测所有单比特错误；
- 检测所有奇数个比特错误；
- 检测所有长度不超过 (r) 的突发错误；
- 对更长突发错误具有很低的漏检概率。

长度为 (r+1) 的突发错误，漏检概率为：

$$\frac{1}{2^{r-1}}$$

长度大于 $(r+1)$ 的突发错误，漏检概率约为：

$$\frac{1}{2^r}$$

突发错误长度

从第一个错误比特到最后一个错误比特之间的总长度，称为突发错误长度；中间的比特不一定全部出错。

八、链路类型

1. 点到点链路

链路只连接两个节点。

例如：

- 主机和交换机端口之间的链路；
- PPP链路；
- 两台路由器之间的专线。

通常不存在多个节点争抢同一链路的问题。

2. 广播链路

多个节点共享同一通信介质。

例如：

- 传统总线型以太网；
- 无线局域网；
- 共享式HFC链路。

多个节点同时发送时，会发生：

冲突 Collision

因此需要多路访问协议协调信道使用。

九、多路访问协议MAC

MAC协议的目标是：

决定多个节点如何共享同一个广播信道。

课件将MAC协议分为三类：

1. 信道划分；
2. 随机访问；
3. 依次轮流。

1. 信道划分协议

把信道资源划分后分配给各节点。

TDMA

按时间划分，每个节点获得固定时隙。

FDMA

按频率划分，每个节点获得固定频段。

CDMA

不同节点使用不同编码。

优点：

- 不发生冲突；
- 高负载时比较稳定。

缺点：

- 某节点没有数据时，其资源可能闲置；
- 低负载时利用率不高。

2. 随机访问协议

特点：

- 不预先划分信道；
- 有数据就尝试发送；
- 允许发生冲突；
- 冲突后恢复和重传。

典型协议：

- ALOHA；
- 时隙ALOHA；
- CSMA；
- CSMA/CD；
- CSMA/CA。

3. 依次轮流协议

节点按顺序获得发送机会。

典型思想：

- 轮询；
- 令牌传递。

优点：

- 不容易冲突；
- 负载较高时效率较好。

缺点：

- 存在控制开销；
- 负责轮询的节点故障可能影响系统；
- 令牌可能丢失。

十、CSMA

CSMA含义是：

载波侦听多路访问

基本思想：

先听后发。

节点准备发送时：

- 信道空闲：发送；
- 信道忙：等待。

为什么CSMA仍会发生冲突

原因是存在：

传播时延

例如A和D相距很远：

1. A开始发送；
2. A的信号还没传播到D；
3. D侦听到的信道仍然空闲；
4. D也开始发送；
5. 两个信号在链路中相遇并冲突。

因此：

即使所有节点都“先听后发”，只要传播时延不为0，仍然可能发生冲突。

传播时延越大，冲突概率通常越高。

十一、CSMA/CD

CSMA/CD是：

载波侦听多路访问/冲突检测

传统共享式有线以太网使用CSMA/CD。

核心思想：

先听后发、边听边发、冲突停止、随机重发

1. 工作过程

第一步：侦听信道

- 空闲：开始发送；
- 忙：继续等待。

第二步：边发送边检测

发送过程中持续检测信号，判断是否发生冲突。

第三步：发现冲突

一旦检测到冲突：

- 立即停止发送；
- 发送Jam强化冲突信号；
- 让所有节点都知道发生了冲突。

第四步：指数退避

随机等待一段时间后重新侦听信道，再尝试发送。

2. 二进制指数退避

第 (m) 次冲突后，从以下整数中随机选择K：

$$K \in 0, 1, \dots, 2^m - 1$$

课件中 (m) 最大取10。

等待时间：

$$T_{\text{backoff}} = K \times 512 \text{ bit time}$$

其中：

$$1 \text{ bit time} = \frac{1}{R}$$

R是链路速率。

因此：

$$T_{\text{backoff}} = K \times \frac{512}{R}$$

3. 各次冲突的K范围

第一次冲突：

$$K \in 0, 1$$

第二次冲突：

$$K \in 0, 1, 2, 3$$

第三次冲突：

$$K \in 0, 1, \dots, 7$$

第十次及以后：

$$K \in 0, 1, \dots, 1023$$

目的：

- 负载低时等待范围较小；
- 负载高、冲突多时扩大随机等待范围；
- 降低再次同时发送的概率。

4. 512比特时间计算

例如100 Mbps以太网：

$$T = \frac{512}{100 \times 10^6}$$

$$T = 5.12 \times 10^{-6} s$$

所以：

$$512 \text{ bit time} = 5.12 \mu s$$

若随机选择 (K=3)：

$$T_{\text{backoff}} = 3 \times 5.12 = 15.36 \mu s$$

十二、CSMA/CD最小帧长计算

为了保证发送方在发送完之前能够检测到最远端发生的冲突，必须满足：

$$T_{\text{frame}} \geq 2\tau$$

其中：

- (T_{frame})：最小帧发送时延；
- (τ)：最远两节点之间的单向传播时延；
- (2τ)：往返传播时延。

帧发送时延：

$$T_{\text{frame}} = \frac{L_{\min}}{R}$$

因此：

$$\frac{L_{\min}}{R} \geq 2\tau$$

最大单向传播时延：

$$\tau_{\max} = \frac{L_{\min}}{2R}$$

如果信号传播速度为 (v)，最大距离：

$$d_{\max} = v\tau_{\max} = \frac{vL_{\min}}{2R}$$

课件例题

100 Mbps局域网，最小帧长128 B。

帧发送时延：

$$T_D = \frac{128 \times 8}{100 \times 10^6}$$
$$T_D = 10.24\mu s$$

它等于最远两节点的往返传播时延，因此最大单向传播时延为：

$$\tau_{\max} = \frac{10.24}{2}$$

$$\tau_{\max} = 5.12\mu s$$

十三、CSMA/CD与CSMA/CA

相同点

二者都基于CSMA：

- 发送前侦听信道；
- 信道空闲才尝试发送；
- 信道忙则等待。

不同点

对比项	CSMA/CD	CSMA/CA
中文	冲突检测	冲突避免
典型环境	有线以太网	无线局域网
工作重点	发生冲突后快速检测并停止	尽量在发送前避免冲突
是否能边发边检测	可以	一般不容易
典型机制	Jam、指数退避	随机退避、ACK、可选RTS/CTS

无线网络中，节点发送时自身信号很强，很难同时检测远处微弱信号，因此采用CSMA/CA，而不是CSMA/CD。

十四、MAC地址

1. 基本结构

MAC地址长度为：

48 bit = 6字节

通常使用十六进制表示：

1A-2F-BB-76-09-AD

或：

1A:2F:BB:76:09:AD

通常每个网络适配器有一个MAC地址。

2. 广播MAC地址

广播地址为：

FF-FF-FF-FF-FF-FF

表示发送给当前局域网中的所有节点。

广播地址只能作为目的MAC地址使用。

3. MAC地址与IP地址区别

对比项	MAC地址	IP地址
层次	链路层	网络层
长度	48位	IPv4为32位
结构	平面地址	层次地址
作用	局域网内帧交付	网络间路由
类比	设备身份证	通信地址
使用范围	当前链路	端到端网络通信

关键理解：

- IP地址告诉数据报“最终去哪里”；
- MAC地址告诉当前这一跳“帧交给谁”。

十五、ARP协议

ARP全称：

Address Resolution Protocol

作用：

已知IP地址，查询对应MAC地址

每台主机和路由器通常维护ARP表：

IP地址	MAC地址	TTL
192.168.1.2	AA-BB-CC-DD-EE-FF	剩余时间

ARP表中的映射不是永久的，经过一段时间会过期。

十六、同一子网内的ARP过程

假设A和B在同一局域网：

- A知道B的IP地址；
- A不知道B的MAC地址。

第一步：查询ARP表

A检查ARP缓存中是否存在：

B的IP → B的MAC

若存在，直接封装以太网帧。

第二步：发送ARP请求

若不存在，A广播ARP请求：

谁拥有IP_B？请告诉IP_A

以太网帧目的MAC：

FF-FF-FF-FF-FF-FF

局域网中所有节点都会收到，但只有IP地址为B的主机会响应。

第三步：ARP应答

B向A发送ARP应答：

IP_B对应的MAC地址是MAC_B

ARP应答通常是单播，目的MAC为A的MAC地址。

第四步：缓存映射

A将：

IP_B → MAC_B

写入ARP表，然后把IP数据报封装成帧：

- 源MAC：MAC_A；
- 目的MAC：MAC_B。

十七、不同子网之间的ARP过程

假设：

- A和B不在同一子网；
- A需要通过路由器R把数据发给B。

A不会ARP查询B的MAC地址，因为B不在本地链路上。

A查询的是：

第一跳：A到路由器R

IP数据报：

- 源IP：IP_A；
- 目的IP：IP_B。

以太网帧：

- 源MAC：MAC_A；
- 目的MAC：MAC_R左接口

路由器收到帧后：

1. 去掉原链路层帧头；
2. 取出IP数据报；
3. 根据目的IP查询路由表；
4. 从另一个接口转发。

第二跳：路由器R到B

路由器在B所在局域网查询B的MAC地址。

重新封装新帧：

- 源MAC：MAC_R右接口；
- 目的MAC：MAC_B；
- 源IP仍然是IP_A；
- 目的IP仍然是IP_B。

必背结论

跨越路由器时：

源、目的IP地址通常不变

但每经过一条链路：

源、目的MAC地址都会重新封装

也就是说：

- IP地址是端到端的；
- MAC地址是逐跳的。

十八、以太网基本特点

以太网是最常用的局域网技术。

早期以太网采用：

- 总线型拓扑；
- 共享信道；
- 半双工；
- CSMA/CD。

现代以太网采用：

- 星型拓扑；
- 中心交换机；
- 点到点链路；
- 全双工；
- 通常不会发生碰撞。

速率不断发展：

10 Mbps → 100 Mbps → 1 Gbps → 10 Gbps及以上

十九、以太网帧结构

课件中的基本结构为：

前导码 | 目的MAC | 源MAC | 类型 | 数据 | CRC

1. 前导码

课件将前导码和帧开始定界符合计为8字节：

- 7字节：10101010重复；
- 1字节：10101011。

作用：

- 让接收方与发送方时钟同步；
- 标志帧即将开始。

2. 目的MAC地址

长度6字节。

接收适配器检查：

- 是否等于自己的MAC；

- 是否为广播地址；
- 是否为需要接收的组播地址。

若都不是，通常丢弃该帧。

3. 源MAC地址

长度6字节，表示发送该帧的接口。

交换机通过源MAC地址进行自学习。

4. 类型字段

指出帧的数据部分交给哪个上层协议，例如：

- IPv4；
- IPv6；
- ARP。

5. 数据字段

通常承载IP数据报。

经典以太网数据字段一般为：

46 ~ 1500字节

如果上层数据不足46字节，需要填充。

6. CRC/FCS

长度通常为4字节。

接收方利用CRC检查帧是否出错：

- 校验通过：接收；
- 校验失败：丢弃。

以太网通常不会在链路层直接纠正错误帧。

7. 最小以太网帧

经典以太网最小帧长：

64字节 = 512 bit

通常指从目的MAC地址到FCS，不包括8字节前导码。

最小帧长与CSMA/CD的冲突检测要求有关。

二十、交换机基本功能

交换机是链路层设备。

主要功能：

- 接收帧；
- 缓存帧；
- 检查目的MAC；
- 查询交换表；
- 选择性转发帧。

交换机通常：

- 即插即用；
- 自动学习；
- 主机无需感知交换机的存在；
- 基本转发不需要配置IP地址。

二十一、交换表

交换机维护交换表：

MAC地址	接口	有效时间
MAC_A	1	9:30
MAC_B	3	9:32

表项含义：

到达某个MAC地址，应从哪个接口转发。

交换表与路由表不同：

- 交换表按MAC地址转发；
- 路由表按IP网络前缀转发。

二十二、交换机自学习

交换机从收到帧的**源MAC地址**学习。

假设交换机从接口1收到帧：

- 源MAC：A；
- 目的MAC：B。

交换机立即记录：

MAC_A → 接口1

因为帧是从接口1进入的，所以A一定可以从接口1到达。

必须牢记：

从源MAC学习，不是从目的MAC学习

二十三、交换机转发规则

交换机收到一个帧后，先学习源MAC，再查询目的MAC。

情况一：目的MAC未知

交换表中没有目的MAC：

泛洪 Flooding

向除入端口以外的所有接口发送。

情况二：目的MAC已知，且接口不同

例如：

目的MAC_B → 接口3

帧从接口1进入，则只从接口3发送：

选择性转发

情况三：目的MAC已知，且接口相同

如果帧从接口1进入，而交换表显示目的主机也在接口1方向：

过滤或丢弃

因为帧无需经过交换机转发回同一网段。

交换机处理流程

收到帧

- 根据源MAC学习或更新交换表
- 查询目的MAC
- 未知：泛洪
- 已知且出端口不同：选择性转发
- 已知且出端口相同：过滤

二十四、交换机与路由器区别

对比项	交换机	路由器
工作层次	链路层	网络层
检查地址	MAC地址	IP地址
转发数据单位	帧	IP数据报
维护表	交换表	路由表/转发表
表项来源	自学习	路由协议或配置
主要范围	同一局域网	不同网络之间
广播处理	通常转发广播	默认不转发二层广播
是否即插即用	通常是	通常需要配置

1. 冲突域

交换机的每个端口通常形成独立冲突域。

现代全双工交换式以太网中，不会发生传统共享介质碰撞。

2. 广播域

普通二层交换机不会隔离广播域。

广播帧会被泛洪到同一VLAN中的其他端口。

路由器可以分隔广播域。